

DAIMO: an integration profile for AI-model governance across dataspaces

Journal Title
XX(X):1–17
©The Author(s) 2026
Reprints and permission:
sagepub.co.uk/journalsPermissions.nav
DOI: 10.1177/ToBeAssigned
www.sagepub.com/

SAGE

Edmundo de Elvira Mori Orrillo¹ and Jiayun Liu¹

Abstract

AI models increasingly circulate between organisations as governed assets, yet publishing a model, selecting it under a usage policy, invoking it through a typed service, tracing each execution, and comparing it under reproducible conditions are five operational tasks that today live in five different vocabularies: MLDCAT-AP describes the model, DCAT publishes it, ODRL expresses the policy, PROV-O records provenance, and the Dataspace Protocol (DSP) negotiates the contractual exchange. No existing ontology connects the five into a single governance chain.

We present DAIMO (*Data-space AI Model Ontology*), an OWL 2 DL integration profile developed under the Linked Open Terms (LOT) methodology. DAIMO reuses the five vocabularies through seven `rdfs:subClassOf` and six `rdfs:subPropertyOf` alignments, and adds nine top-level native classes plus five role subclasses to cover the bridge constructs that no reused vocabulary provides: the offering record, the execution authorisation, the model deployment, the derived artefact, the cross-participant provenance bundle, the I/O contract, the audit evidence, and the shared evaluation context.

DAIMO ships three complementary evidence mechanisms. Consistency reasoning over the union of core and alignments is complemented by an *entailment-verification check* that tests every inferred superclass of every native class against a list of semantically forbidden ones. A SHACL layer combines nine structural shapes, three conformance shapes over reused classes, and six cross-class SHACL-SPARQL invariants; each invariant is paired with a crafted violating case, and the negative test bench fires all six. An illustrative multi-participant demonstrator instantiates the profile, exercises every native class, and supports twenty-three competency questions elicited during requirements specification. The artefact is distributed as a public package with ontology, SHACL layer, demonstrator graph, SPARQL queries, validation scripts, and WIDOCO documentation.

Keywords

AI model governance; dataspace; ontology alignment; SHACL validation; LOT methodology

Introduction

Regulation (EU) 2024/1689, the *AI Act*, requires for high-risk AI systems technical documentation of the model, a record of each execution, traceability of training, and audit evidence sustained across the full lifecycle³⁵. In parallel, the European dataspace initiative, materialised in Eclipse Dataspace Components (EDC)³³ connectors and sectoral deployments such as INESData and Gaia-X, establishes a contractual exchange model between autonomous participants on top of the Dataspace Protocol (DSP)³². These two frameworks, taken together, place AI models in a new regime: they must be able to circulate between organisations as governed assets, with actionable metadata that sustain publishing, discovering, invoking, tracing, and comparing them coherently. Semantic-Web vocabularies cover each of these five tasks in isolation, and this article argues that integrating them into a single profile is both necessary and non-trivial.

MLDCAT-AP 3.0.0 (SEMIC, 2025)³¹ extends DCAT towards machine learning. An AI model becomes a catalog asset with task, algorithm, runs, evaluations, risks, and compute resources as first-class metadata. DCAT² supplies the catalog primitives (`Catalog`, `Dataset`, `Distribution`, `DataService`, `CatalogRecord`)

that MLDCAT-AP rests on. ODRL⁴ provides a mature model for offers, permissions, prohibitions, and agreements, and is the natural substrate for the usage policies that govern invocation. PROV-O³ is the standard for provenance and attribution across entities, activities, and agents. DSP³² specifies the contract-negotiation and transfer messages that EDC implements. Model cards⁶, factsheets⁷, and datasheets¹⁹ complement these vocabularies with narrative descriptions of intended use, limitations, and training conditions.

Each vocabulary solves its part. The gaps between them are the problem. A model published as `it6:MachineLearningModel` carries no attached act of being offered across an organisational boundary. An `odrl:Offer` is not anchored to a concrete model inside a federated catalog. A DSP negotiation is asset-agnostic: it does not know that the exchange concerns an AI model

¹Department of Artificial Intelligence, ETSI Informáticos, Universidad Politécnica de Madrid, Spain

Corresponding author:

Edmundo de Elvira Mori Orrillo, Universidad Politécnica de Madrid, Campus de Montegancedo, 28660 Boadilla del Monte, Madrid, Spain.

Email: edmundo.mori.orrillo@upm.es

whose metrics and evaluation contexts condition its use. A `prov:Bundle` does not, by itself, group activities that occurred under distinct administrative contexts around the same derived artefact. Publishing, discovering, invoking, tracing, and comparing AI models in a dataspace requires these gaps to be articulated coherently.

No reusable ontology currently links catalog publication metadata, access policies, invocation contracts, execution traces, audit evidence, and comparative evaluation for AI models circulating between participants of a dataspace, with axiomatic alignments verified against existing vocabularies and with executable, reproducible evidence of correctness. Closing that distance demands two simultaneous principles: we must reuse what the mature vocabularies already solve, and we must add only the constructs that the articulation between them requires.

We present DAIMO (*Data-space AI Model Ontology*) and contribute three elements, supported by a reproducible illustrative demonstrator.

The **first** is the integration profile itself: an OWL 2 DL ontology that reuses DCAT-AP, MLDCAT-AP 3.0.0, ODRL 2.2, PROV-O, and DSP through seven `rdfs:subClassOf` and six `rdfs:subPropertyOf` alignments rather than bare prefix declarations. On top of that reuse layer, DAIMO introduces fourteen native classes that cover the bridge constructs absent from the union of the reused vocabularies. Each native class is justified by a concrete gap and by at least one competency question it answers. The delta over MLDCAT-AP is deliberately narrow: DAIMO reuses the AI-asset plane directly and adds only the governance bridge between the catalog and the dataspace.

The **second** is an *entailment-verification* mechanism over the OWL-RL closure that complements standard consistency reasoning. For each native class, the check enumerates the inferred superclasses and tests them against a list of semantically forbidden ones. This catches alignments that a DL reasoner accepts without contradiction yet introduces unintended typings under reasoning.

The **third** is a SHACL layer that bundles six cross-class SPARQL invariants with a negative test bench. Each invariant comes with a purpose-built violating case. Validation conforms on the positive graph and fires all six violations on the negative graph, showing that the constraints are discriminating and not merely satisfiable.

As supporting evidence, we instantiate the profile over an illustrative multi-participant scenario with five anonymised actors (a model provider, a municipal consumer, a scientific evaluator, a federated dataspace platform, and a compliance office). The demonstrator exercises every native class at least once and supports all twenty-three competency questions. It is a reproducibility artefact, not a deployed pilot.

Background and related work

DAIMO sits at the intersection of three threads of work that the literature typically treats in isolation: describing AI models (narrative and structured), governing resources and dataspace, and executable ontology engineering in the *Semantic Web Journal*. We review each thread in turn, identifying what it contributes to the catalog-dataspace

bridge, where it leaves the bridge open, and how DAIMO positions itself against it. Table 2, inspired by the reuse matrix of RePlanIT⁹, compares the first two threads against the five operational attributes that any profile for governed AI models must cover.

Describing AI models

Descriptions of AI artefacts appear at two levels of formality. *Narrative* documentation—model cards⁶, factsheets⁷, and datasheets for datasets¹⁹—raised transparency standards by describing intended use, limitations, performance on benchmarks, and training conditions in natural language. These formats are valuable for human readers and qualitative accountability, but insufficient as a substrate for automated discovery, policy-based filtering, or evidence linking in machine-to-machine flows.

Structured *ontologies* of the ML lifecycle cover the machine-consumable layer. EXPO²² offers a general ontology of scientific experiments that later work has built on. ML-Schema²³ formalises machine-learning experiments as a reusable, lightweight schema. MEX²⁴ provides an interchange vocabulary, and OntoDM²⁵ together with DMOP²⁶ organise the data-mining domain. Souza et al.²⁷ and Schmidt et al.²⁸ extend the perspective towards provenance and FAIRness across the lifecycle. These ontologies offer a rigorous foundation but remain centred on the scientific experiment and the training process; none addresses the contractual dimension of exchange between autonomous participants.

The work closest to DAIMO is MLDCAT-AP 3.0.0³¹ (SEMIC, 2025). MLDCAT-AP extends DCAT to describe AI models as subclasses of `dc:Dataset` with rich metadata on policy, benchmarks, evaluation, execution, risk, and compute infrastructure. DAIMO reuses it directly: `it6:MachineLearningModel` is the model class, and `it6:Task`, `it6:Run`, `it6:Flow`, `it6:Evaluation`, and `it6:ComputerInfrastructure` respectively capture the target task, the concrete execution, the associated pipeline, the resulting measurement, and the compute environment. MLDCAT-AP is strong on the model side but agnostic to the dataspace in which the model circulates: it does not model the act of offering a model across an organisational boundary, nor governed deployment, authorisation anchored to an agreement, or cross-participant provenance aggregation. MLDCAT-AP covers the model; DAIMO covers the bridge between the model and the dataspace.

Table 1 makes the MLDCAT-AP relationship explicit. For each MLDCAT-AP class most likely to co-occur with DAIMO, it records whether DAIMO reuses the class directly, imposes an additional conformance shape on it, or complements it with a new bridge class that MLDCAT-AP does not cover. The pattern is consistent: on the AI-asset plane DAIMO reuses and does not redefine; on the catalog-dataspace bridge DAIMO adds without overriding.

Governing resources and dataspace

Two threads of work provide the governance substrate that DAIMO composes on top of the model description. The first is the family of W3C *governance vocabularies* for resources. DCAT 2² provides the catalog primitives

Table 1. Relationship between DAIMO and MLDCAT-AP 3.0.0, class by class. *Reuse direct* means DAIMO uses the class unchanged. *Conformance* means DAIMO imposes a SHACL shape requiring a subset of properties on the class. *Complement* means DAIMO introduces a new native class that MLDCAT-AP does not cover.

MLDCAT-AP class	DAIMO treatment	Notes
it6:MachineLearningModel	Reuse + conformance	Reused as the AI asset; MachineLearningModelInDAIMOShape adds mandatory policy, title, identifier.
it6:Task	Reuse direct	Target task of a model; used as is.
it6:Run	Reuse + conformance	RunInDAIMOShape requires flow, algorithm, agent, timestamp; daimo:authorizedBy links runs to authorisations.
it6:Flow	Reuse direct	Pipeline associated with a run.
it6:Evaluation	Reuse direct	Measurement result; aggregated into CrossParticipantProvenanceRecord via daimo:records.
it6:EvaluationMeasure	Reuse direct	Metric type and value.
it6:Benchmark	Reuse direct	Benchmark reference; cited from SharedEvaluationContext.
it6:ComputerInfrastructure	Reuse direct	Compute environment on which a deployment runs.
it6:Hardware,	Reuse direct	Elements of the compute environment.
it6:Library		
it6:HarmRisk	Reuse direct	Risk annotation on a model.
it6:Modality	Reuse direct	Input modality.
it6:trainedOn,	Reuse direct	Training/test dataset links.
it6:testOn		
Bridge constructs absent from MLDCAT-AP and added by DAIMO		
(no MLDCAT-AP analogue)	Complement	daimo:AIAssetOffering – catalog record with attached ODRL policy, aligned to dcat:CatalogRecord.
(no MLDCAT-AP analogue)	Complement	daimo:ExecutionAuthorization – ODRL agreement that authorises concrete it6:Runs.
(no MLDCAT-AP analogue)	Complement	daimo:ModelDeployment – typed link between model and dcat:DataService.
(no MLDCAT-AP analogue)	Complement	daimo:IOContract – I/O formats and authentication method.
(no MLDCAT-AP analogue)	Complement	daimo:DerivedArtifact – governable output of an authorised run.
(no MLDCAT-AP analogue)	Complement	daimo:AuditEvidence – integrity record with spdx:Checksum.
(no MLDCAT-AP analogue)	Complement	daimo:CrossParticipantProvenanceRecord – PROV bundle aggregating activities across edc:ParticipantContexts.
(no MLDCAT-AP analogue)	Complement	daimo:SharedEvaluationContext – reified comparability conditions.
(no MLDCAT-AP analogue)	Complement	daimo:ParticipantRole + 5 subclasses – role in dataspace exchange.

on which MLDCAT-AP and DAIMO rest. ODRL 2.2⁴ offers a consolidated model for permissions, prohibitions, obligations, offers (odrl:Offer), and binding agreements (odrl:Agreement); DAIMO uses it extensively for the policy attached to offerings and for the agreements that authorise concrete executions. PROV-O³ provides the provenance ontology over activities, entities, agents, bundles, and roles, and is the natural substrate for tracing individual executions and their cross-participant aggregation. Pandit and Esteves¹⁵ show how ODRL combines with DPV for health-data scenarios; DAIMO transposes that pattern to the AI domain. These vocabularies are mature and widely adopted but domain-agnostic: none is specific to AI, and none is specific to dataspace.

The second thread is the emerging set of *dataspace standards* that describe the exchange itself. Franklin et al.¹ framed *dataspaces* as a response to persistent heterogeneity, and Otto et al.¹⁷ and Cappiello et al.¹⁸ turn data sovereignty, contracts, and controlled interoperability into

explicit design assumptions. The Dataspace Protocol³² is the W3C-CG specification that defines catalog, contract-negotiation, and transfer-process messages; Eclipse Dataspace Components³³ is its reference implementation. One distinction deserves emphasis: EDC is a Java execution framework, not an ontology. The semantics that EDC implements come from DSP (protocol), ODRL (policies), and DCAT (catalog); only a handful of concepts are genuine EDC-specific extensions, and of those DAIMO explicitly references edc:ParticipantContext to anchor aggregated provenance to an administrative context. Work on federated ecosystems such as ConSolid⁸ further shows that the Semantic Web is evolving towards multi-actor scenarios where governance and interoperability must be designed jointly.

Executable ontology engineering in SWJ

Beyond the domain substrate, a third thread of recent work shapes how DAIMO should be evaluated. The *Semantic Web Journal* has recently published a body of ontologies that share an explicit methodological commitment to executable validation and reproducibility. Kurteva et al. present RePlanIT⁹, an ontology for digital product passports of ICT equipment, with SHACL and industry-expert usability evaluation. Palma et al. operationalise soil information at global scale in GloSIS¹⁰ through a modular profile aligned with SOSA/SSN and ISO 28258. Woods et al. connect ontology engineering with natural-language processing for industrial maintenance activities¹¹. Bareedu et al. derive SHACL validation rules from OPC UA industry standards¹³. Ferranti et al. formalise Wikidata property constraints through SHACL and SPARQL¹⁴. Robaldo and Batsakis¹⁶ examine the interplay between SHACL validation and inference on the Time Ontology. Penteado et al.¹² study methodologies for publishing linked open government data. Collectively, these works set the expectation that an operationally oriented ontology ships reproducible evidence of correctness, not merely an axiomatic description. DAIMO meets that expectation.

Methodology and requirements

We adopted the Linked Open Terms (LOT) methodology²⁹ for two reasons. First, its industrial orientation: LOT prescribes a lightweight workflow with deliverables at each of its four phases (requirements specification, implementation, publication, and maintenance). Second, LOT treats reuse as a first-class phase rather than a by-product, which matches the reuse-by-default discipline the problem demands. The subsections below describe the executive scenario that drove requirements elicitation (§), the competency questions and non-functional requirements that resulted (§), and the three core design decisions that shaped the ontology (§).

Illustrative scenario

Requirements were elicited from an illustrative scenario in which five anonymised actor types exchange a geospatial risk-prediction model across a federated dataspace: *Provider A* (research organisation publishing the model), *Consumer B* (municipal consumer invoking predictions), *Platform C* (operator of the federated dataspace), *Evaluator D* (scientific evaluator publishing comparative measurements), and *Compliance E* (governance body auditing the chain). The scenario synthesises the five operational tasks a catalog-dataspace bridge must support: publishing, discovering, invoking, tracing, and evaluating. Section describes the instantiation in detail. The scenario is a reproducibility demonstrator rather than a deployed pilot; replacing it by a real pilot with documented partners, or by an instantiation over a public MLDCAT-AP catalog, is a coherent next step.

Requirements specification

We elicited requirements from three complementary sources: the illustrative scenario of §, the public specification of MLDCAT-AP 3.0.0, and the public documentation of

Eclipse Dataspace Components and the Dataspace Protocol. From these sources we formulated twenty-three competency questions, organised into five categories by actor and lifecycle stage. Category R (registration and publication, five questions) covers the provider's need to publish a model as a governed asset with identity, licence, policy, and invocation contract. Category D (discovery and selection, four questions) captures the questions a consumer asks to locate a suitable model by task, domain, policy, and quality threshold. Category E (execution and auditability, five questions) groups the questions that a platform operator and a governance actor ask in order to invoke services, trace executions, and reconstruct evidence. Category V (evaluation and reproducibility, five questions) contains the evaluator's questions about comparing results under homogeneous conditions. Finally, category G (governance bridge, four questions) covers the multi-actor questions whose answer requires exclusively DAIMO-native semantics and, as we argue in §, cannot be answered with the reused vocabularies alone. Table 3 summarises the actor-category mapping; the full set of twenty-three questions in natural language appears in Appendix .

Eleven of the twenty-three questions require explicit reasoning (subclass paths, subproperty chains, or SPARQL aggregation); the remaining twelve are direct queries over DAIMO-native data. This proportion is deliberate. A profile resting exclusively on direct queries would not exercise the part of the artefact that makes OWL semantics valuable, and a profile demanding reasoning for every question would be impractical on real endpoints.

Non-functional requirements fix the engineering constraints. The ontology targets the OWL 2 DL profile for decidability and HermiT compatibility, ships under CC-BY 4.0 (Apache 2.0 for the validation scripts), and uses persistent URIs under <https://w3id.org/pionera/daimo> with content negotiation and accessible SHACL validation. A reuse-by-default rule binds the modelling work: every DAIMO term must justify why none of DCAT, DCAT-AP, MLDCAT-AP, ODRL, PROV-O, DSP, or the EDC extension covers it.

Core design decisions

Three structural decisions shape the rest of the design.

Decision 1: reuse preferentially. We reuse MLDCAT-AP classes directly whenever MLDCAT-AP already models the concept precisely. The model itself maps to `it6:MachineLearningModel`, the runtime environment to `it6:ComputerInfrastructure`, and the experimental triad of execution, pipeline, and measurement to `it6:Run`, `it6:Flow`, and `it6:Evaluation`. Publication and policy are similarly covered by existing vocabularies: `dcat:Dataset`, `dcat:Distribution`, and `dcat:DataService` on the DCAT side, `odrl:Offer` and `odrl:Agreement` on the ODRL side.

Introducing DAIMO-native equivalents for any of these would violate Gruber's principle of minimal ontological commitment and fracture the DCAT-AP ecosystem without adding expressive power. A native class is therefore justified only when the union of the reused vocabularies leaves a

Table 2. Reuse matrix: families of related work against the five operational attributes that a profile for governed AI models must cover. ● = natively covered; ○ = partially covered via extensions; — = not covered.

Family / artefact	Publication	Policy	Invocation	Traceability	Evaluation
Model cards, factsheets, datasheets ^{6,7,19}	○	—	—	—	○
ML-Schema, MEX, OntoDM, DMOP ^{23–26}	—	—	—	○	●
MLDCAT-AP 3.0.0 ³¹	●	○	—	●	●
DCAT 2 ²	●	—	○	—	—
ODRL 2.2 ⁴	—	●	—	—	—
PROV-O ³	—	—	—	●	○
DSP / EDC ^{32,33}	○	○	●	—	—
DAIMO (this work)	●	●	●	●	●

Table 3. Actors involved in the executive scenario and the competency-question categories derived from them.

Actor	Category	Need	CQs
Model provider	R	Publish a model as a governed asset with identity, licence, policy, and invocation contract	CQ-R1–CQ-R5
Model consumer	D	Discover models by task, domain, policy, and quality threshold	CQ-D1–CQ-D4
Platform operator	E	Invoke services, trace executions, and reconstruct operational evidence	CQ-E1–CQ-E5
Evaluator	V	Compare results under a shared context and inspect reproducibility	CQ-V1–CQ-V5
Governance (multi-actor)	G	Bridge offer, deployment, authorisation, and cross-participant provenance	CQ-G1–CQ-G4

concrete gap that prevents answering at least one competency question.

Decision 2: separate three complementary planes. The architecture distinguishes three planes that DAIMO connects but does not replace. Plane A is the dataspace plane: DSP and the specific EDC extensions DAIMO references. Plane B is the platform governance plane: INESData or an analogous platform, with its federated registration and operational policies. Plane C is the AI/ML asset plane: MLDCAT-AP and the vocabularies that support it. DAIMO is the integration profile that connects A, B, and C; it is not a fourth parallel layer.

Decision 3: one native class per gap, not per theme. Given reuse preference, each DAIMO-native class must justify its existence by a concrete gap in the union of the reused vocabularies and by at least one competency question that cannot be answered without it. Strict application of this criterion produces nine top-level classes (AIAssetOffering, ParticipantRole, ModelDeployment, IOContract, ExecutionAuthorization, DerivedArtifact, CrossParticipantProvenanceRecord, AuditEvidence, and SharedEvaluationContext) plus five role subclasses. Any candidate class failing both conditions simultaneously is discarded.

The DAIMO ontology

Modular architecture and metrics

Five tasks define the scope: publication as a catalog record, policy-and-quality-sensitive discovery, invocation controlled by typed services, traceability of individual executions, and contextualised and reproducible evaluation. Orthogonal responsibilities (training-pipeline detail, fine-grained

privacy-policy semantics, formal PROV derivation theory) already belong to mature vocabularies whose duplication would fracture the ecosystem. DAIMO adds the constructs that make those vocabularies articulate coherently when an AI model circulates between autonomous participants.

Three Turtle files decompose the ontology so that a consumer can load only what they need. The **core module** (`daimo-core.ttl`) contains the fourteen native classes, twenty-nine object properties and eight datatype properties, the global disjointness axioms, and the property characterisations (functional, asymmetric, inverse) that capture TBox-level type invariants. The **alignment module** (`alignment.ttl`) encapsulates the seven `rdfs:subClassOf` and six `rdfs:subPropertyOf` declarations that connect DAIMO to the reused vocabularies, plus minimal declarations of the external terms with `rdfs:seeAlso` pointers to their source specifications. The **SHACL module** (`daimo-shapes.ttl`) contributes eighteen `sh:NodeShapes`: nine structural, three of conformance over reused classes, and six cross-class invariants via `sh:SPARQLConstraint`. Because the module is declared as an independent `owl:Ontology` with its own `owl:versionIRI`, the validation layer has its own lifecycle.

Table 4 summarises the size and expressivity of the artefact, extracted directly from the Turtle files and from the reports produced by the validation suite (ġ). The numbers let a reader compare DAIMO with previously published ontologies in the same family.

[TODO: A complementary OntoMetrics-style table (Protégé plugin) is missing, breaking axiom counts down by type: SubClassOf, EquivalentClasses, DisjointClasses, ObjectPropertyDomain, ObjectPropertyRange, FunctionalObjectProperty, AsymmetricObjectProperty, InverseObjectProperties, SubDataPropertyOf,

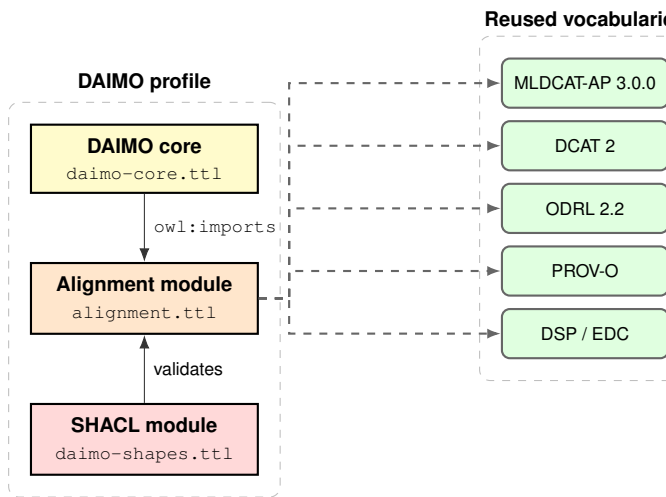


Figure 1. Modular architecture of DAIMO and reused vocabularies. Yellow, orange, and red boxes denote the three DAIMO modules (core, alignment, shapes); green boxes denote the reused vocabularies. Dashed arrows from the alignment module carry the `rdfs:subClassOf` and `rdfs:subPropertyOf` declarations. The core module imports alignments for inference; the SHACL module validates data against the combined TBox.

Table 4. DAIMO artefact metrics verified against package files.

Dimension	Value
Declared logical profile	OWL 2 DL
Native top-level classes	9
Native subclasses (ParticipantRole)	5
Total DAIMO-native classes	14
Object properties (owl:ObjectProperty)	29
Datatype properties (owl:DatatypeProperty)	8
Functional properties (owl:FunctionalProperty)	28
Asymmetric properties (owl:AsymmetricProperty)	5
Explicit inverse pairs (owl:inverseOf)	5
<code>rdfs:subClassOf</code> to externals	7
<code>rdfs:subPropertyOf</code> to externals	6
Named disjointness axiom	1
Total <code>sh:NodeShapes</code>	18
completeness (one per DAIMO class)	9
conformance over reused classes	3
cross-class SPARQL invariants	6
Competency questions	23
OOPS! pitfalls (Critical/Important/Minor)	0 / 0 / 2

DataPropertyDomain, DataPropertyRange, AnnotationAssertion, etc. RePlanIT §5.2 and most SWJ ontology papers report this table; it gives the reviewer a quantitative measure of axiomatic richness independent of class and property counts.]

Reused vocabulariesNative classes and alignments

The nine top-level native classes, together with the five ParticipantRole subclasses, form the minimal set that the three core design decisions produce. Table 5 synthesises each class with its axiomatic alignment to the reused vocabularies; the subsections that follow examine those that carry the most semantic load. Figure 2 shows the overall view of classes and native properties.

The two classes without external alignment, IOContract and SharedEvaluationContext, lack one deliberately. IOContract is not a `dcatalog:DataService` (a service is an accessible resource, a URL, an endpoint, whereas the contract describes syntactic and authentication expectations over the exchange) and it is not a `dcat:Standard` (the contract enumerates concrete obligations, not a reference to an external specification). SharedEvaluationContext is a methodological reification, not a task: it captures the exact combination under which two evaluations are comparable, not the abstract task those evaluations instantiate. Aligning it to `owl:Thing` or `prov:Entity` would add noise without adding semantics.

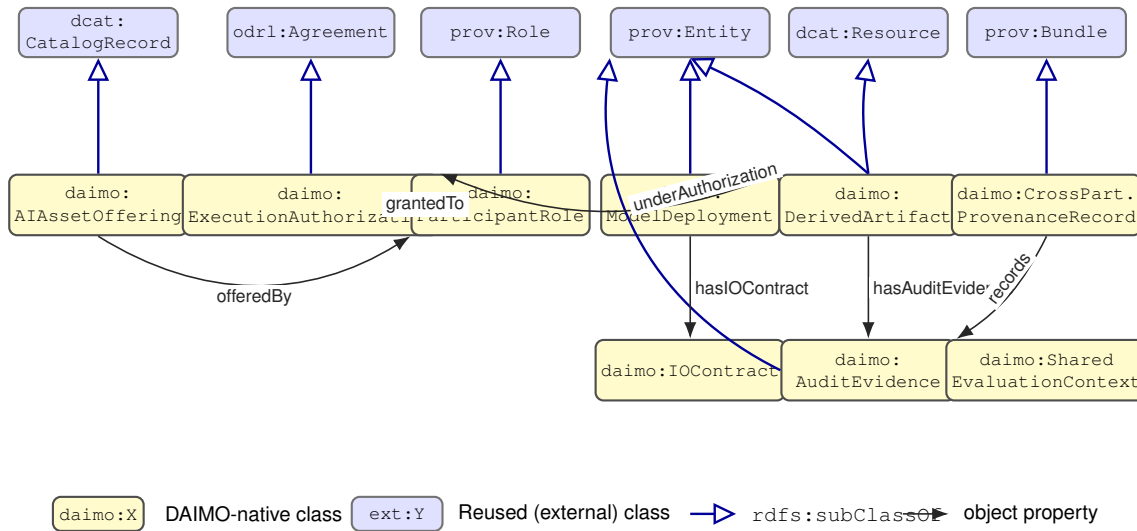
In the executive scenario of §, the provider publishes its models as three AIAssetOffering instances in the federated catalog; each offering links the model through `daimo:offersModel`, the publisher through `daimo:offeredBy`, and the ODRL policy through `daimo:hasOfferPolicy`. The deployment of the model selected by the municipal consumer is modelled as a ModelDeployment that, through `daimo:exposedAs`, points to two parallel `dcatalog:DataServices` (REST and gRPC), each with its own IOContract. The authorisation the consumer obtains from the provider after DSP negotiation is reified as ExecutionAuthorization and linked, through `daimo:authorizesRun`, to the specific executions it permits. The prediction produced by each execution is reified as DerivedArtifact with its associated AuditEvidence, and the CrossParticipantProvenanceRecord aggregates the full chain across the `edc:ParticipantContexts` of the provider, the consumer, and the federated platform.

Registration and participation. AIAssetOffering and ParticipantRole jointly solve the publication-side gap. MLDCAT-AP describes the model; it does not reify the *act* of offering it across an organisational boundary, and DCAT’s Catalog/CatalogRecord pair is agnostic to the governance context that the offering carries. AIAssetOffering $\sqsubseteq$ `dcatalog:CatalogRecord` is the catalog-record of a model in a specific dataspace, attaching an ODRL policy through `daimo:hasOfferPolicy`, the published model through `daimo:offersModel` (aligned to `foaf:primaryTopic`), and its publisher through `daimo:offeredBy`. ParticipantRole, with its five non-disjoint subclasses ModelProvider, ModelConsumer, PlatformOperator, Evaluator, and ComplianceOfficer, reifies the role a legal agent plays in relation to an asset and a dataspace context. The two classes jointly answer CQ-R1, CQ-R2, CQ-R5, and CQ-G1.

Execution plane. ModelDeployment and IOContract cover the invocation-side gap. DCAT supplies

Table 5. DAIMO-native classes, axiomatic alignments, and the competency questions they answer.

Class	Alignment	Role	CQs
daimo:AIAssetOffering	dcat:CatalogRecord	Catalog record that publishes a model as a governed offer in a dataspace; carries an ODRL policy via daimo:hasOfferPolicy.	R1, R2, R5, G1
daimo:ParticipantRole (+5 non-disjoint subclasses)	prov:Role	Role a participant plays with respect to an AI asset within a dataspace context.	R5
daimo:ModelDeployment	prov:Entity	Running or hosted instance of a model, exposed as a dcat:DataService over an it6:ComputerInfrastructure.	E1, E3, G2
daimo:IOContract	(no external alignment)	Minimal actionable contract: I/O formats, authentication method, optional schemas.	R4, E1, G2
daimo:ExecutionAuthorization	odrl:Agreement	ODRL agreement derived from a DSP negotiation that authorises concrete executions via daimo:authorizesRun.	E2, E5, G3
daimo:DerivedArtifact	prov:Entity, dcat:Resource	Governed and catalogable output produced by an execution under authorisation.	E5, G4
daimo:CrossParticipantProvenanceRecord	prov:Bundle	Aggregation of activities and entities across participant contexts forming an audit narrative.	G4
daimo:AuditEvidence	prov:Entity	Compliance-oriented evidence record with structured spdx:Checksum, signer, and timestamp.	E4
daimo:SharedEvaluationContext	(no external alignment)	Reified comparability conditions: task, dataset, version, protocol, and random seed.	V1, V2, V3

**Figure 2.** Overview of the DAIMO native classes (yellow) with their axiomatic alignments to the reused vocabularies (blue) via `rdfs:subClassOf` (open-triangle arrows), and the main native properties that articulate the profile (labelled object-property arrows). *DerivedArtifact* has two external parents, `prov:Entity` and `dcat:Resource`; *AuditEvidence* is placed in the bottom row but is also a `prov:Entity`. Classes *IOContract* and *SharedEvaluationContext* carry no external alignment by design. Properties whose targets are classes of the reused vocabularies (*offersModel* to *it6:MachineLearningModel*, *authorizesRun* to *it6:Run*) are declared in Table 5 and are omitted here for readability.

`DataService` as a catalog-visible endpoint; it does not tie a service to the model it serves, nor does it expose the syntactic and authentication expectations that a consumer must satisfy to invoke the service. `ModelDeployment` $\sqsubseteq$ `prov:Entity` is the running or hosted instance of an `it6:MachineLearningModel` on an `it6:ComputerInfrastructure`. It is linked to the model through `daimo:deploysModel` (asymmetric) and exposed through `daimo:exposedAs` to one or several `dcat:DataServices`. Each service carries a `daimo:hasIOContract` to an `IOContract`, which enumerates I/O format, authentication method (restricted by

`sh:in` to a closed vocabulary of nine tokens), and optional schema references. Both properties are non-functional, so a deployment may announce several services and contracts in parallel. This group answers CQ-E1, CQ-E3, and CQ-G2.

Authorisation and derivation. *ExecutionAuthorization* and *DerivedArtifact* cover the authorisation-to-output chain. ODRL supplies *Agreement* as a binding contract; PROV-O supplies *wasGeneratedBy/wasDerivedFrom* for provenance; neither, on its own, records the specific act of authorising a concrete `it6:Run` or

the governance-visible artefact that the run produces. `ExecutionAuthorization` $\sqsubseteq$ `odrl:Agreement` is the DSP-negotiated agreement that binds specific ODRL permissions to specific runs through the asymmetric property `daimo:authorizesRun`. Its inverse `daimo:authorizedBy` is functional: each run has at most one covering authorisation. `DerivedArtifact` $\sqsubseteq$ `prov:Entity`, `dcat:Resource` is the governance-visible output of a run, linked back to the run via `daimo:derivedFromRun` and to the authorisation via `daimo:underAuthorization`. INV-1 enforces the consistency of this chain. The group answers CQ-E2, CQ-E5, CQ-G3, and CQ-G4.

Audit and provenance aggregation. `AuditEvidence` and `CrossParticipantProvenanceRecord` cover the auditability-across-boundaries gap. SPDX supplies `Checksum`; PROV-O supplies `Bundle`; neither addresses compliance-oriented evidence that crosses administrative contexts. `AuditEvidence` $\sqsubseteq$ `prov:Entity` bundles an `spdx:Checksum` (promoted to a named node for stable external reference), a signer, and a timestamp; the property `daimo:evidenceOf` links it asymmetrically to the derived artefact or run it supports. `CrossParticipantProvenanceRecord` $\sqsubseteq$ `prov:Bundle` aggregates activities and entities across multiple `edc:ParticipantContexts` via `daimo:spansContext` and `daimo:records`, producing an audit narrative that a governance actor can traverse end-to-end. The group answers CQ-E4, CQ-G4.

Evaluation context. `SharedEvaluationContext` covers the comparability gap that plain `it6:Evaluation` leaves open. MLDCAT-AP records a measurement attached to a model; it does not reify the combination of task, dataset version, evaluation protocol, and random seed that makes two measurements comparable. `SharedEvaluationContext` is a methodological reification of those four coordinates: it carries `daimo:onTask`, `daimo:onDatasetVersion`, `daimo:protocol` (restricted by `sh:pattern` to canonical protocol names), and `daimo:seed`. Two `it6:Evaluation` instances sharing a `SharedEvaluationContext` are comparable by construction; instances that differ on any coordinate are not. The class answers CQ-V1, CQ-V2, CQ-V3.

Defending the heaviest alignments. Two of the seven `rdfs:subClassOf` alignments carry significantly more semantic weight than the others. Their counter-readings are plausible and will be raised by any specialised reviewer; we defend each below.

`AIAssetOffering` $\sqsubseteq$ `dcat:CatalogRecord`, *not* `dcat:Dataset`. DCAT carefully distinguishes the offered resource (the `dcat:Dataset`, the thing of interest) from the record that describes that resource in a concrete catalog (the `dcat:CatalogRecord`, the metadata artefact). We align `AIAssetOffering` to `dcat:CatalogRecord` because an offering is conceptually the cataloguing record of the model in a specific dataspace, with a policy and a set of metadata specific to that context. The offering is not the model itself, which is

reused directly as `it6:MachineLearningModel`. A counter-reading that subsumed `AIAssetOffering` under `dcat:Dataset` would conflate two distinct entities: the model (which can appear in several catalogs with different conditions) and its offering in one specific catalog (which is one and has its own policy). This distinction is operationally relevant: the same `it6:MachineLearningModel` may be referenced by several `AIAssetOfferings` with distinct policies, and the property `daimo:offersModel`, aligned to `foaf:primaryTopic`, captures that the offering *is about* the model without being identified with it.

`ExecutionAuthorization` $\sqsubseteq$ `odrl:Agreement`, *not* `prov:wasDerivedFrom odrl:Agreement`. An execution authorisation is the specific result of a DSP negotiation that binds concrete ODRL permissions to concrete invocations, and ODRL models precisely this kind of binding instrument as `odrl:Agreement`: the equivalent of a contract signed between an assigner and an assignee, carrying effective permissions and restrictions. The counter-reading that would relate the authorisation to the agreement via `prov:wasDerivedFrom` treats the authorisation as a distinct artefact from the agreement, introducing a distinction ODRL does not need: an Agreement is already the binding entity. The subsumption uses the full ODRL semantics on permissions, prohibitions, and obligations without redefining it, and allows existing ODRL tools to treat DAIMO authorisations as standard agreements. What DAIMO adds over `odrl:Agreement` is the asymmetric property `daimo:authorizesRun`, which links the agreement to the `it6:Runs` it covers: a relation ODRL does not prescribe but that the bridge to the PROV plane requires.

The remaining five alignments are briefly defended below, ordered from the most to the least contentious.

`ParticipantRole` $\sqsubseteq$ `prov:Role`. PROV-O scopes Role to a qualified association inside a single activity, while the DAIMO role is scoped to a dataspace context and typically outlives any single execution. The subsumption is therefore defensible only in the *is-a-kind-of* sense: a DAIMO role *is* a role in the PROV sense whenever a specific activity qualifies it, and the scope difference is captured by the additional property `daimo:inContext` linking the role to an `edc:ParticipantContext`. A less committal alternative would be `skos:related` to `prov:Role`, which we rejected because it would prevent reasoners from classifying DAIMO roles as PROV roles when they do qualify an association. We accept the tighter subsumption with the understanding that contexts extending beyond a single activity are an additional constraint, not a contradiction.

`ModelDeployment` $\sqsubseteq$ `prov:Entity`. A deployment is a persistent artefact over which activities happen (model loading, inference, logging), not an activity itself. Modelling it as `prov:Activity` would force each deployment to carry `startedAtTime/endedAtTime`, which fits a run but not a long-lived hosted instance. Aligning it to `prov:Entity` lets the native property `daimo:deploysModel` act as a genuine entity-to-entity link, and preserves the intuition that running a model on

infrastructure produces a new governable entity rather than a new activity.

`DerivedArtifact` $\sqsubseteq$ `prov:Entity`, `dcat:Resource`. The double subsumption reflects two complementary roles: a derived artefact is a *provenance entity* produced by a run, and simultaneously a *catalogable resource* that a governance actor may list, describe, and distribute. `prov:Entity` and `dcat:Resource` are compatible: DCAT itself treats `dcat:Resource` as the top of its resource hierarchy with no commitment against PROV. The OWL-RL closure over the demonstrator confirms that no unintended typing emerges from the double parent: the entailment-verification check inspects every inferred superclass of `DerivedArtifact` and raises zero warnings. The double alignment is what allows a single artefact to be cited both from a provenance bundle (as `prov:Entity`) and from a DCAT catalog record (as `dcat:Resource`).

`CrossParticipantProvenanceRecord` $\sqsubseteq$ `prov:Bundle`. PROV-O offers two collection constructs: `prov:Bundle` (a named set of provenance descriptions with its own attribution) and `prov:Collection` (a set of entities with no attached provenance meta-semantics). The record is exactly the first: an attributable named grouping of provenance descriptions across several `edc:ParticipantContexts`. Aligning to `prov:Collection` would lose the meta-provenance layer that makes audit narratives inspectable (a bundle itself has a `wasAttributedTo` agent and a `generatedAtTime`); aligning to both would introduce redundancy. `prov:Bundle` is the right choice.

`AuditEvidence` $\sqsubseteq$ `prov:Entity`. Evidence is a produced artefact (a hash value plus its signer and timestamp), not the act of producing it. The auditing activity is a separate `prov:Activity` instance associated with the compliance officer; `AuditEvidence` is what that activity generates. Aligning the class to `prov:Activity` would conflate the two and break INV-4-like invariants that compare authorisation validity against evidence generation time. The choice of `prov:Entity` keeps the activity-entity distinction that PROV relies on.

Axiomatic foundations

A taxonomy alone does not pin down meaning; it only names the types. Behind the DAIMO taxonomy sits an axiomatic infrastructure that fixes what each class is, what it is not, and how its instances must behave. Three complementary strata make up this infrastructure: OWL axioms over native classes and properties, SHACL nodal constraints over instances, and SHACL-SPARQL cross-class invariants. The first stratum is described below; the SHACL strata appear in § as part of validation.

On the OWL stratum, disjointness between the nine native top-level kinds is declared through the named axiom `daimo:TopLevelKindsDisjointness`, an instance of `owl:AllDisjointClasses`. The five `ParticipantRole` subclasses are deliberately excluded: one legal agent may hold several roles at once (provider and operator, for instance), which reflects that roles are context-dependent rather than mutually exclusive.

The disjointness core is complemented by property characterisations. Twenty-eight properties are declared functional (`owl:FunctionalProperty`) on those whose domain is a class with a nodal shape, so that maximum cardinality is axiomatised simultaneously in the logical layer and in the SHACL layer. Five key properties (`offersModel`, `deploysModel`, `authorizesRun`, `derivedFromRun`, and `evidenceOf`) are marked asymmetric, which formally captures that a deployment is not the model it deploys, an authorisation does not coincide with the execution it authorises, and so on. Five explicit inverse pairs (`authorizedBy` $\leftrightarrow$ `authorizesRun`, `hasDeployment` $\leftrightarrow$ `deploysModel`, `hasDerivedArtifact` $\leftrightarrow$ `derivedFromRun`, `hasAuditEvidence` $\leftrightarrow$ `evidenceOf`, `hasOffering` $\leftrightarrow$ `offersModel`) enable native inverse queries without forcing the publisher of the graph to materialise both directions.

The SHACL stratum splits into two families. Each DAIMO-native class carries a structural nodal shape that prescribes which properties are mandatory in a well-formed instance; these nine shapes are the syntactic guarantee that a publisher has supplied everything the semantic invariants need. Three conformance shapes (`OfferInDAIMOShape`, `MachineLearningModelInDAIMOShape`, and `RunInDAIMOShape`) establish which properties of reused classes are mandatory when a graph declares adherence to the DAIMO profile. The distinction between the two families is intentional: DAIMO does not override the underlying ontology, because that would fracture the ecosystem. Instead, it declares a commitment to a concrete subset of properties on the external classes, making the adherence criterion explicit without modifying the original definitions.

Above the shapes sit the six SHACL-SPARQL cross-class invariants (INV-1 through INV-6), examined in detail in §. They are the stratum that sets DAIMO apart from a purely descriptive profile: they encode rules whose meaning cannot be expressed as a local constraint on a single shape, because the rules span several classes simultaneously. Listing 1 shows how the OWL axioms are encoded for the concrete case of `ExecutionAuthorization`: the subclass of `odrl:Agreement`, the asymmetry of `authorizesRun`, the functionality of its inverse, and membership in the named disjointness axiom.

```
daimo:ExecutionAuthorization a owl:Class ;
  rdfs:subClassOf odrl:Agreement ;
  skos:definition "An ODRL agreement produced by
    a DSP contract
    negotiation that binds specific permissions
    to run invocations..."@en .

daimo:authorizesRun a owl:ObjectProperty , owl:
  AsymmetricProperty ;
  rdfs:domain daimo:ExecutionAuthorization ;
  rdfs:range it6:Run .

daimo:authorizedBy a owl:ObjectProperty , owl:
  FunctionalProperty ;
  owl:inverseOf daimo:authorizesRun ;
  rdfs:domain it6:Run ;
  rdfs:range daimo:ExecutionAuthorization .
```

```

daimo:TopLevelKindsDisjointness a owl:
  AllDisjointClasses ;
  owl:members ( daimo:AIAssetOffering
    daimo:ExecutionAuthorization
    daimo:ModelDeployment
    daimo:DerivedArtifact
    daimo:IOContract
    daimo:AuditEvidence
    daimo:
      CrossParticipantProvenanceRecord
    daimo:SharedEvaluationContext
    daimo:ParticipantRole ) .

```

Listing 1: Core axioms around ExecutionAuthorization.

Reuse surface. Table 6 summarises the reused vocabularies and their concrete use. Three observations matter. First, the alignments to DCAT and ODRL target the standard vocabularies, not internal entities of any implementation—a distinction frequently blurred in Eclipse EDC practice. Second, DAIMO maintains an informative mapping relation toward DSP through `skos:closeMatch` and `skos:related`, not a subsumption one: DSP is a messaging protocol, not an asset ontology, and subsuming DAIMO classes under DSP constructs would introduce spurious typings. Third, of the entire EDC extension only `edc:ParticipantContext` appears explicitly in DAIMO, referenced to anchor aggregated provenance.

Three subproperties that might seem natural (`authorizesRun` $\sqsubseteq$ `prov:used`; `grantedTo` $\sqsubseteq$ `prov:qualifiedAssociation`; `evidenceOf` $\sqsubseteq$ `prov:hadActivity`) are deliberately not declared, because each would produce unintended typings on authorisations, agents, or evidence under RDFS closure. The entailment-verification check of § catches errors of this kind.

Evaluation

Evaluation follows a four-dimension pattern adapted from RePlanIT⁹. We first instantiate the ontology over the executive scenario as a case study (§). We then verify the formal correctness of the ontology through consistency reasoning, entailment checking, pitfall scanning, and alignment verification (§). Next, we validate instance-level data against the SHACL layer (§) and confirm that the twenty-three competency questions return their expected rows (§). Every dimension is reproducible from the public package through `validate.py`, `reasoner_check.py`, `oops_check.py`, and `tests/negative_test.py`.

Illustrative demonstrator

The scenario introduced in § is instantiated over DAIMO as a reproducibility demonstrator. The resulting graph (`examples/scenario.ttl`) contains 225 data triples that exercise at least once every native class, and over which the twenty-three competency questions of § are run. All identifiers are anonymised placeholders (`ex:provider-a`, `ex:consumer-b`, and so on); model name, dataset version, and metric values are synthetic and are declared as such. The demonstrator witnesses the articulation between classes; it does not claim deployment evidence.

Provider A publishes three `AIAssetOffering` instances inside catalog `ex:provider-a-catalog` (a `dcat:Catalog` with `dcat:record` triples pointing to the three offerings). The first, `ex:offering-model-alpha-v2`, registers model `ex:model-alpha-v2`; the second, `ex:offering-model-alpha-v1`, registers baseline `ex:model-alpha-v1`; the third, `ex:offering-model-alpha-v2-ro`, registers a variant of the v2 model under a more restrictive policy. All three offerings share the same `it6:Task` and domain but differ in ODRL policy: the baseline offering and the v2 offering carry a permissive policy with an `odrl:use` permission, while the third offering adds an `odrl:prohibition` on commercial use.

Model `ex:model-alpha-v2` is deployed (as a `ModelDeployment` instance, `ex:deployment-model-alpha-v2`) on an `it6:ComputerInfrastructure` described through framework, accelerator, and library identifiers. The deployment is exposed as two simultaneous `dcat:DataServices` with two `IOContracts`: a REST endpoint with JSON input and `oauth2-bearer` authentication, and a gRPC endpoint with protobuf and `mtls`. The multi-endpoint usage explicitly exercises the modelling decision of treating `daimo:exposedAs` and `daimo:hasIOContract` as non-functional properties: one deployment may announce several services and contracts at once, to cover distinct authentication profiles.

Consumer B obtains an `ExecutionAuthorization` linked to a specific `odrl:use` permission with explicit validity dates. Execution `ex:run-consumer-b` is an `it6:Run` that `prov:wasAssociatedWith` Consumer B and is authorised via `daimo:authorizesRun`. The execution produces a `DerivedArtifact` and an `AuditEvidence` whose `daimo:integrityHash` points to a named `spdx:Checksum` with SHA-256 algorithm and explicit digest. Promoting the checksum to a named node, rather than a blank node, enables stable external references from file systems and logging.

A `CrossParticipantProvenanceRecord` aggregates the entire chain across three `edc:ParticipantContexts` corresponding to Provider A, Consumer B, and Platform C. Evaluator D publishes two `it6:Evaluation` instances with synthetic accuracy values, both sharing a `SharedEvaluationContext` fixed to a common benchmark, protocol, and seed. Figure 3 illustrates the instances and their relations.

Formal verification

Both reasoners agree. HermiT, invoked through `owlready2`, confirms that the union of the core, the alignments, and the example graph is logically consistent with no unsatisfiable classes; reasoning takes 0.72 s. OWL-RL materialisation (the `owlrl` library) derives 1 171 additional triples on top of 817 initial ones (1 988 in total) in 0.27 s, with no individuals typed as `owl:Nothing` and no unsatisfiable subclasses.

Logical consistency is necessary but not sufficient. An ontology may be consistent and still, for instance, infer

Table 6. Vocabularies reused by DAIMO and the concrete use of each.

Vocabulary	Reused terms	Use in DAIMO
MLDCAT-AP 3.0.0 (it6:)	MachineLearningModel, Task, Run, Flow, Evaluation, EvaluationMeasure, Benchmark, ComputerInfrastructure, Hardware, Library, HarmRisk, Modality, trainedOn, testedOn	AI-asset plane; DAIMO does not redefine the model.
DCAT 2 (dcat:)	Catalog, Dataset, Distribution, DataService, CatalogRecord, endpointURL	Publication and discovery.
ODRL 2.2 (odrl:)	Offer, Agreement, Permission, Prohibition, hasPolicy, target, assigner, assignee	Policy, offer, and agreement.
PROV-O (prov:)	Activity, Entity, Agent, Bundle, Role, wasGeneratedBy, wasAssociatedWith	Provenance and roles.
SPDX (spdx:)	Checksum, algorithm, checksumValue	Audit-evidence integrity.
DSP (dsp:)	ContractOffer, ContractNegotiation, TransferProcess	Informative skos:closeMatch/skos:related mappings; not subsumption.
EDC (edc:)	ParticipantContext	Only EDC-specific extension referenced.

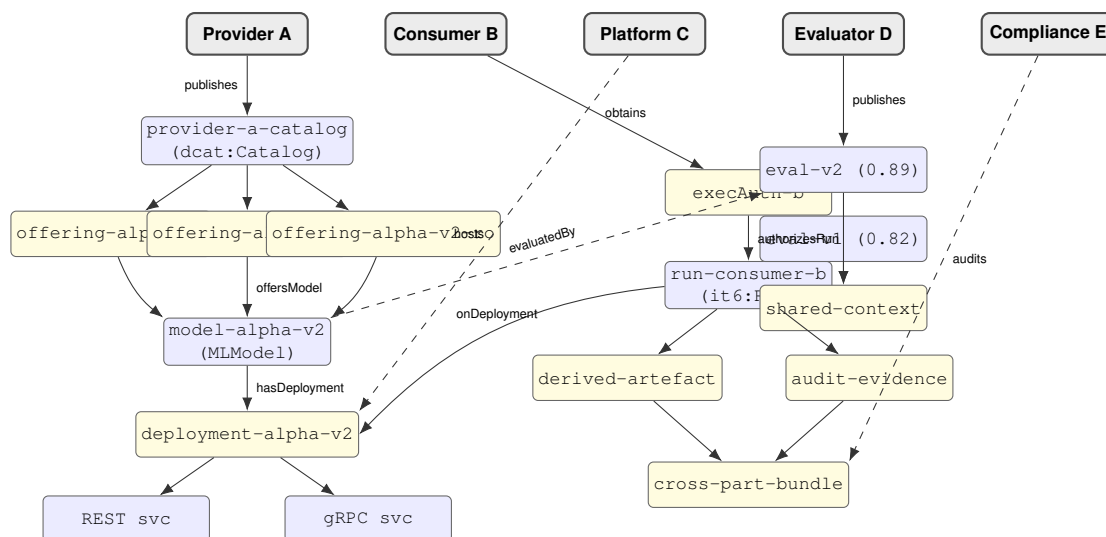


Figure 3. Illustrative demonstrator instantiated over DAIMO with five anonymised participants. Grey rounded boxes on the top row are the five participants (one per `edc:ParticipantContext`); yellow-shaded instances are DAIMO-native (offerings, deployments, authorisations, derived artefacts, audit evidence, cross-participant bundle, shared evaluation context); light-blue instances are reused classes (catalog, model, services, runs, evaluations). Solid arrows denote direct properties; dashed arrows denote the participant's functional role over the instance below.

that an `AuditEvidence` is a `prov:Activity`—a thing produced by activities, not an activity. That is why we added an entailment-verification check over the OWL-RL closure: for each native class, the script `reasoner_check.py` enumerates every inferred superclass and tests it against a list of forbidden ones. The report covers the fourteen native classes (nine top-level plus five `ParticipantRole` subclasses) and raises no warning. This check is complementary to consistency: it catches alignment errors that a DL reasoner accepts because they are logically consistent yet semantically wrong.

The OOPS! scanner³⁰, queried through its REST API on the union of the core and alignments, reports zero

Critical pitfalls, zero Important ones, and two Minor ones: P13 (“inverse relationships not declared”) on 34 elements with no semantically meaningful inverse, and P04 (“isolated ontology elements”) on seven locally declared external classes. Both arise from the reuse-by-default discipline the profile applies.

The final layer of formal verification targets the alignment module itself. Seven `rdfs:subClassOf` and six `rdfs:subPropertyOf` declarations connect DAIMO to its five parent vocabularies (DCAT, MLDCAT-AP, ODRL, PROV-O, and FOAF). The entailment-verification check inspects the OWL-RL closure of each alignment and confirms that none produces a semantically incorrect

superclass on any DAIMO-native class, raising zero warnings over the fourteen classes. We deliberately excluded three subproperty alignments toward PROV-O from the module for the same reason: each would have produced a forbidden typing under RDFS closure (ĝ).

SHACL validation: structural, invariants, and negative test bench

DAIMO layers three SHACL *shape* families, each answering a different validation question. Structural shapes ask whether every DAIMO-native instance carries the mandatory properties of its class. Conformance shapes ask whether reused external classes satisfy the additional obligations that the profile imposes on them. SPARQL invariants ask whether relations across several classes remain coherent. Together, the three families move validation from syntactic to semantic: from “is this instance well-formed?” to “is this graph internally consistent?”.

The nine structural shapes impose minimum completeness per DAIMO class, and the 225-triple positive graph satisfies `conforms=True`. The three conformance shapes add operational obligations on reused classes: `OfferInDAIMOShape` requires every `odrl:Offer` used in DAIMO to declare `odrl:assigner` and `odrl:target` (at policy level or per permission, through `sh:or`); `MachineLearningModelInDAIMOShape` requires an ODRL policy, title, and identifier on `it6:MachineLearningModel`; `RunInDAIMOShape` requires flow, algorithm, associated agent, and timestamp on `it6:Run`. Additionally, `sh:in` restricts `daimo:authMethod` values to a closed vocabulary of nine tokens (`none`, `api-key`, `basic`, `bearer`, `oauth2-bearer`, `oauth2-client-credentials`, `mtls`, `jwt`, `dataspace-token`), and `sh:pattern` restricts `daimo:protocol` to a pattern accepting the usual evaluation-protocol names (`holdout`, `stratified-holdout`, `leave-one-out`, `temporal-split`, `<n>-fold-cv`, `stratified-<n>-fold-cv`, `bootstrap-<n>`).

Above this structural stratum sit the six SHACL-SPARQL cross-class invariants, encoded through `sh:SPARQLConstraint`. They are the differentiator against a purely descriptive profile. INV-1 guarantees consistency of the derivation-authorisation chain: every `DerivedArtifact` must carry a `daimo:underAuthorization` property whose authorisation covers precisely the execution the artefact is derived from. INV-2 forces the agent associated with an execution through `prov:wasAssociatedWith` to also appear as `daimo:grantedTo` on one of the authorisations covering that execution, making explicit that the PROV-registered actor and the ODRL-authorized beneficiary must be the same entity. INV-3 checks data-plane coherence: the `dcat:DataService` exposed by a `ModelDeployment` must serve exactly the model the deployment declares, ruling out an endpoint that advertises one model and serves another. INV-4 requires `daimo:expiresAt` of every `ExecutionAuthorization` to be strictly later than `prov:startedAtTime` of every execution the authorisation covers, reflecting that an authorisation

cannot justify executions that started after it lapsed. INV-5 requires the model referenced by `daimo:offersModel` in an `AIAssetOffering` to appear as `odrl:target` of the attached policy, at policy level or in one of its permissions; if the policy does not govern the registered model, the governance chain is broken even if both resources exist. INV-6 closes the same triangle on the provider side: `daimo:offeredBy` of every offering must equal `odrl:assigner` of its policy, preventing inconsistencies between the catalog-record attribution and the ODRL offer’s declared authorship. The positive graph conforms to all six invariants without warnings.

Listing 2 shows the concrete encoding of INV-5 as `sh:SPARQLConstraint`, illustrating the `FILTER NOT EXISTS` pattern that captures the disjunction between policy-level and per-permission targets.

```
daimo:OfferingPolicyTargetInvariant a sh:NodeShape
;
sh:targetClass daimo:AIAssetOffering ;
sh:sparql [
  a sh:SPARQLConstraint ;
  sh:message "INV-5 violation: offering's
    offersModel does not
    appear as odrl:target of the
    attached policy."@en ;
  sh:prefixes daimo:_invariantPrefixes ;
  sh:select """
    SELECT $this ?model ?policy WHERE {
      $this daimo:offersModel ?model ;
      daimo:hasOfferPolicy ?policy .
      FILTER NOT EXISTS { ?policy odrl:
        target ?model }
      FILTER NOT EXISTS { ?policy odrl:
        permission/odrl:target ?model }
    }
    """ ;
]
```

Listing 2: INV-5 encoded as a `sh:SPARQLConstraint`.

Shapes that conform on the positive graph are necessary but not sufficient to show that constraints are semantically discriminating. A trivially satisfiable constraint on any graph would pass positive validation without yielding real guarantees. For this reason DAIMO ships a companion graph `tests/negative-examples.ttl` (118 triples) that encodes six deliberately invalid cases (`bad:INV1-artifact`, `bad:INV2-run`, `bad:INV3-deployment`, `bad:INV4-auth`, `bad:INV5-offering`, and `bad:INV6-offering`), each crafted to violate exactly one invariant. The `tests/negative_test.py` script loads the ontology, the shapes, and the negative graph, runs SHACL, and verifies two conditions: global validation fails, and every expected focus node appears in the report with the specific invariant’s message. All six invariants fire correctly on their designated violation. This evidence complements positive validation and is what allows us to claim, with grounding, that the invariants capture real business rules.

Competency questions

The twenty-three competency questions are expressed as SPARQL queries (`queries/queries.md`) and run over the OWL-RL materialised closure of the positive graph.

Table 7 summarises coverage by category, observed row counts, and queries requiring explicit reasoning; the full set of queries appears in Appendix . All twenty-three return at least one row, which confirms that the demonstration graph contains the information needed to answer every question with the primitives DAIMO declares.

Two observations deserve emphasis. CQ-R2 and CQ-E1 depend on the `subPropertyOf` relation between `daimo:offersModel` and `foaf:primaryTopic` and return no rows on a plain SPARQL engine without reasoning. The dependency is documented rather than defective: aligning `offersModel` under `foaf:primaryTopic` makes offerings discoverable through the standard DCAT property, a benefit that materialises only under inference. A consumer who must run those queries on a plain endpoint can materialise the closure beforehand or rewrite the queries to enumerate both properties. The second observation is that CQ-G2 returns two rows, one per endpoint of the multi-protocol REST and gRPC deployment. This confirms that modelling several services per deployment is a legitimate, directly SPARQL-observable pattern.

Two fragments illustrate the pattern. Listing 3 shows CQ-G3 (“which authorisation authorised a specific run, and which ODRL agreement backs it?”): the query traverses the `daimo:authorizedBy` inverse from an `it6:Run` to an `ExecutionAuthorization`, then lifts the agreement’s assigner and assignee. Listing 4 shows CQ-V3 (“ranking of models under the same `SharedEvaluationContext` and metric”): the query joins evaluations against a fixed context, orders by metric value, and binds the model and the run that produced each evaluation. Both queries run over the OWL-RL-materialised closure of the demonstrator; the full twenty-three SPARQL queries are published in `queries/queries.md` for inspection.

```
SELECT ?auth ?assigner ?assignee WHERE {
  ?run a it6:Run ;
    daimo:authorizedBy ?auth .
  ?auth a daimo:ExecutionAuthorization ;
    odrl:assigner ?assigner ;
    odrl:assignee ?assignee .
  FILTER (?run = <https://example.org/daimo-
    scenario/run-consumer-b>)
}
```

Listing 3: CQ-G3: authorisation and agreement for a specific run.

```
SELECT ?model ?metric ?value WHERE {
  ?eval a it6:Evaluation ;
    daimo:hasEvaluationContext ?ctx ;
    it6:hasMeasure ?m .
  ?m it6:metricType ?metric ;
    it6:metricValue ?value .
  ?run it6:hasEvaluation ?eval ;
    daimo:onDeployment/daimo:deploysModel ?
    model .
  FILTER (?ctx = <https://example.org/daimo-
    scenario/eval-context>)
} ORDER BY DESC(?value)
```

Listing 4: CQ-V3: ranking of models under a shared evaluation context.

Discussion

DAIMO connects publication, policy, execution, provenance, and evaluation in a single profile. That articulation sustains governed AI-model exchange across organisational boundaries, and it distinguishes DAIMO both from recent domain ontologies (RePlanIT⁹, GloSIS¹⁰, Woods et al.¹¹) and from earlier machine-learning lifecycle ontologies. With the former, DAIMO shares an engineering ethos of reuse, executable implementation, and CQ-based evaluation, but applies it to the catalog-dataspace bridge. With the latter, DAIMO shares the application domain but adds the governance layer they do not address. Three design features set the artefact apart. First, entailment verification over the OWL-RL closure complements consistency reasoning: it enumerates inferred superclasses of each native class and detects semantically wrong alignments a DL reasoner accepts without contradiction. Second, cross-class SHACL-SPARQL invariants are paired with a negative test bench; every invariant fires on a case built to violate it, not only on the positive graph. Third, reuse is axiomatic, not nominal: seven explicit `rdfs:subClassOf` and six `rdfs:subPropertyOf` declarations, plus three deliberately undeclared subproperties whose adoption would have introduced unintended typings.

Three lines of argument bound the limitations. The first concerns *semantic depth*. `IOContract` captures I/O format and authentication method (restricted by `sh:in` to a closed vocabulary) but not fine-grained semantic compatibility between model and consumer schemas, which is delegated to `it6:Flow` as Kipoi²⁰ and TFX²¹ do for pre-processing and pipelines. `SharedEvaluationContext` reifies task-dataset-version-protocol-seed for single-metric comparisons; heterogeneous benchmarking protocols and composite metric sets remain open. The role model folds role type (`ModelProvider`) into role assignment (the agent-role-context tuple), which `OntoClean` recommends splitting. `MLDCAT-AP` folds them the same way in `hasRegisteredUser`, which places fine-grained contextualisation in the future-refinement horizon, not at the level of a present design error.

The second line concerns the *integration commitments* and their effects on consumers who combine DAIMO with adjacent vocabularies. Property `daimo:records` declares `rdfs:range prov:Activity`; under RDFS reasoning, `MLDCAT-AP` evaluations grouped inside a `CrossParticipantProvenanceRecord` are inferred as `PROV` activities even though `MLDCAT-AP` does not declare that subsumption. The effect is harmless for audit queries, but a consumer combining DAIMO, `MLDCAT-AP`, and `PROV-O` may see surprising typings on `it6:Evaluation`. Relatedly, DAIMO is not anchored to an upper ontology (BFO, GFO, DOLCE, UFO). The absence is intentional, because an integration profile is not a reference ontology. A coherent future step would be to accommodate the taxonomy to one of these upper ontologies without giving up direct reuse of the domain vocabularies that motivate DAIMO.

The third line concerns *reproducibility conditions and external validation*. Reported numerical results depend on a specific reasoning profile: `inference=rdfs` mode in

Table 7. Coverage of the 23 competency questions. Each CQ returns the indicated row count over the positive graph under OWL-RL closure. Boldface marks queries that require explicit reasoning beyond plain SPARQL.

Category	Queries (rows)	N	Inference required
Registration and publication (R)	R1(3) R2(1) R3(1) R4(2) R5(2)	5	R2 (subPropertyOf chain), R5 (subClassOf path)
Discovery and selection (D)	D1(3) D2(2) D3(4) D4(1)	4	D1 (task subtype), D2 (policy matching)
Execution and auditability (E)	E1(4) E2(1) E3(2) E4(1) E5(1)	5	E1 (offersModel $\sqsubseteq$ foaf:primaryTopic), E2 (agreement-run union)
Evaluation and reproducibility (V)	V1(1) V2(1) V3(2) V4(1) V5(4)	5	V2, V3 (aggregation)
Governance bridge (G)	G1(1) G2(2) G3(1) G4(1)	4	G3 (subClassOf reasoning), G4 (GROUP_CONCAT aggregation)

pyshacl, necessary for implied subproperties to materialise before the conformance shapes are evaluated, and the OWL-RL closure materialised before the competency queries. This dependency is documented but is a condition a consumer must know to reproduce exact results. Expert-based validation is not part of the evidence reported here; a qualitative study with three practitioner profiles (an EDC or INESData engineer, an MLDCAT-AP/SEMIC maintainer, and an MLOps practitioner) is the next phase of work. **[TODO: Run the expert study: recruitment, semi-structured interview protocol, qualitative analysis, and publication of results.]**

Adoption and envisaged integration paths are concrete even though a deployed pilot is not part of the present evidence. On the AI-asset side, DAIMO is designed to co-evolve with the SEMIC MLDCAT-AP dossier: it reuses the 3.0.0 vocabulary directly and adds no classes that overlap the MLDCAT-AP scope, which keeps a MLDCAT-AP-native consumer compatible without translation. On the dataspace side, the natural integration host is Eclipse Dataspace Components: DAIMO references `edc:ParticipantContext` explicitly to anchor aggregated provenance, and its ODRL agreements are standard `odrl:Agreement` instances that an EDC connector already produces at the end of a DSP negotiation. On the governance side, DAIMO’s `AuditEvidence` class is structured to carry the integrity records that a Gaia-X Trust Framework inspection, or an AI Act Annex IV traceability audit, would request: checksum, signer, timestamp, and linkage to a cross-participant bundle. We envisage three concrete validation paths for adoption: instantiation over a public MLDCAT-AP catalog extract (to verify the reuse claims under real metadata), co-deployment with an EDC connector on a federated pilot (to witness the DSP-to-ExecutionAuthorization chain end-to-end), and mapping of a high-risk AI Act use case against the six SHACL-SPARQL invariants (to assess operational fit). None of these has been executed yet; they define the short-term adoption roadmap rather than the present claims.

The relation between DAIMO and MLDCAT-AP deserves explicit delimitation. The model itself is reused directly through `it6:MachineLearningModel`. DAIMO’s contribution concentrates on the catalog-dataspace bridge: the four competency questions of the governance category exercise exclusively DAIMO-native classes and cannot be answered with MLDCAT-AP alone, because they require `AIAssetOffering` (CQ-G1), `ModelDeployment`

(CQ-G2), `ExecutionAuthorization` (CQ-G3), and `CrossParticipantProvenanceRecord` (CQ-G4). The two profiles are complementary.

Looking beyond the present artefact, DAIMO illustrates a pattern that is likely to recur wherever autonomous participants need to exchange high-value, policy-bound resources. The pattern has three recurring elements: an offering record reified above the resource itself, an authorisation reified above the permission, and a provenance bundle that crosses administrative contexts. Genomic datasets, clinical decision-support tools, digital twins, and industrial models all present analogous publication-invocation-auditing chains. DAIMO is AI-specific in what it reuses (MLDCAT-AP) but domain-agnostic in the constructs it adds, and we hope the constructs will serve as a starting point for sibling profiles in adjacent domains.

Availability and FAIR compliance. DAIMO is distributed as a public package under CC-BY 4.0 (ontology and documentation) and Apache 2.0 (validation scripts). The artefact is *findable* through the persistent `w3id.org/pionera/daimo` namespace with perversion IRIs and Dublin Core metadata; *accessible* through content negotiation across Turtle, RDF/XML, JSON-LD, and N-Triples; *interoperable* through exclusive reuse of W3C-standard vocabularies and OWL 2 DL profile; and *reusable* through SKOS definitions and examples per term, SemVer versioning (with a deprecation policy based on `owl:deprecated` and `skos:closeMatch`), and SHACL shapes that act as a data contract. Appendix lists the deliverable items and their locations, and the full release procedure is documented in `DEPLOYMENT.md` so that any future maintainer can run it without implicit knowledge.

Conclusions

DAIMO’s delta over the reused vocabularies is the governance bridge between the AI catalog and the dataspace: offering, authorisation, deployment, derived artefact, and cross-participant provenance. The four governance-category competency questions witness the gap concretely; they cannot be answered with any of the reused vocabularies alone. Entailment verification, a negative SHACL-SPARQL test bench, and the reproducible demonstrator provide three complementary mechanisms of evidence. Future work follows from the bounded limitations: external expert validation,

instantiation over a public MLDCAT-AP catalog, refinement of IOContract toward schema compatibility, role-type/role-assignment split following OntoClean, and extension of SharedEvaluationContext to heterogeneous benchmarking protocols. The package is published under CC-BY 4.0 (ontology) and Apache 2.0 (scripts) and is ready to be adopted, extended, or challenged by the community.

References

- Franklin, M., Halevy, A. and Maier, D. (2005) From Databases to Dataspace: A New Abstraction for Information Management. *SIGMOD Record*, 34(4):27–33.
- Albertoni, A. et al. (2020) *Data Catalog Vocabulary (DCAT) — Version 2*. W3C Recommendation.
- Lebo, P., Sahoo, S. and McGuinness, D. (eds.) (2013) *PROV-O: The PROV Ontology*. W3C Recommendation.
- Iannella, R., Guth, S. and Steyskal, R. (eds.) (2018) *ODRL Information Model 2.2*. W3C Recommendation.
- Knublauch, H. and Kontokostas, D. (eds.) (2017) *Shapes Constraint Language (SHACL)*. W3C Recommendation.
- Mitchell, M. et al. (2019) Model Cards for Model Reporting. *Proceedings of the Conference on Fairness, Accountability, and Transparency*.
- Arnold, M. et al. (2019) FactSheets: Increasing Trust in AI Services through Supplier’s Declarations of Conformity. *IBM Research Report*.
- Werbrouck, J. et al. (2024) ConSolid: A federated ecosystem for heterogeneous multi-stakeholder projects. *Semantic Web*, 15:429–460.
- Kurteva, A. et al. (in press) RePlanIT Ontology for Digital Product Passports of ICT: Laptops and Data Servers. *Semantic Web*.
- Palma, R. et al. (in press) GloSIS: The Global Soil Information System Web Ontology. *Semantic Web*.
- Woods, C. et al. (in press) An ontology for maintenance activities and its application to data quality. *Semantic Web*.
- Penteado, B.E., Maldonado, J.C. and Isotani, S. (2023) Methodologies for publishing linked open government data on the Web. *Semantic Web*, 14:585–610.
- Bareedu, Y.S. et al. (2024) Deriving semantic validation rules from industrial standards: An OPC UA study. *Semantic Web*, 15:517–554.
- Ferranti, N. et al. (in press) Formalizing and Validating Wikidata’s Property Constraints using SHACL and SPARQL. *Semantic Web*.
- Pandit, H.J. and Esteves, B. (in press) Enhancing Data Use Ontology (DUO) for Health-Data Sharing by Extending it with ODRL and DPV. *Semantic Web*.
- Robaldo, L. and Batsakis, S. (in press) On the interplay between validation and inference in SHACL. *Semantic Web*.
- Otto, B. et al. (2019) Reference architecture model for international data spaces. *International Journal of Information Management*, 45:182–199.
- Cappiello, C., Gal, A., Jarke, M., Rehof, J. and Yang, Y. (2022) Data spaces: Design, deployment, and future directions. *ACM Computing Surveys*, 55(2):Article 46.
- Gebru, T. et al. (2021) Datasheets for datasets. *Communications of the ACM*, 64(12):86–92.
- Avsec, Z. et al. (2019) The Kipoi repository accelerates community exchange and reuse of predictive models for genomics. *Nature Biotechnology*, 37.
- Baylor, D. et al. (2017) TFX: A TensorFlow-based production-scale machine learning platform. *KDD*.
- Soldatova, L.N. and King, R.D. (2006) An ontology of scientific experiments. *Journal of the Royal Society Interface*, 3(11).
- Correa Publio, G. et al. (2018) ML-Schema: Exposing the semantics of machine learning with schemas and ontologies.
- Keet, C.M., Fernández, J.D. and Panov, P. (2016) MEX vocabulary: A lightweight interchange format for machine learning experiments. *ISWC*.
- Panov, P., Dzeroski, S. and Soldatova, L.N. (2008) OntoDM: An ontology of data mining. *ICDMW*.
- Keet, C.M. et al. (2015) The data mining optimization ontology. *Web Semantics*, 32.
- Souza, R. et al. (2019) Provenance data in the machine learning lifecycle in computational science and engineering. *WORKS*.
- Schmidt, M. et al. (2021) FAIRness along the machine learning lifecycle using Dataverse in combination with MLflow. *ICDE Workshops*.
- Poveda-Villalón, M., Fernández-Izquierdo, A., Fernández-López, M. and García-Castro, R. (2022) LOT: An industrial oriented ontology engineering framework. *Engineering Applications of Artificial Intelligence*, 111:104755.
- Poveda-Villalón, M., Gómez-Pérez, A. and Suárez-Figueroa, M.C. (2014) OOPS! (Ontology Pitfall Scanner!). *International Journal on Semantic Web and Information Systems*, 10(2):7–34.
- SEMIC (2025) *MLDCAT-AP 3.0.0: Machine Learning Data Catalogue Application Profile*. European Commission.
- International Data Spaces Association (2024) *Dataspace Protocol (DSP)*. W3C Community Draft.
- Eclipse Foundation. *Eclipse Dataspace Components (EDC)*, control-plane documentation.
- Garijo, D. (2017) WIDOCO: A Wizard for Documenting Ontologies. *ISWC*.
- European Union (2024) *Regulation (EU) 2024/1689 laying down harmonised rules on artificial intelligence (AI Act)*. Official Journal of the European Union.

Artefact availability

The reproducibility workflow does not require prior deployment of the persistent URL. Any user with Python 3.12 and the declared dependencies (`rdflib`, `pyshacl`, `owlrl`, `owlready2`) can run the four scripts locally and reproduce every report in `reports/`. The persistent URL and the DOI are additional FAIR requirements that complement but do not condition reproducibility.

Competency questions

R — Registration and publication (5)

- CQ-R1: Which AI models has a given provider published as governed catalog assets?
- CQ-R2: For a given model, which catalog offering registered it, by which publisher, and when? (requires `subPropertyOf chain`)

Table 8. Deliverable items of the DAIMO public package and their locations.

Item	Location
Persistent namespace	https://w3id.org/pionera/daimo [TODO: merge the PR on perma-id/w3id.org so that storing a model for a metrics via content negotiation.]
Versioned modules	Three independent ontologies https://w3id.org/pionera/daimo/ with explicit owl:priorVersion and owl:versionIRI and
Public repository	[TODO: Create public GitHub repository pionera/daimo by running DEPLOYMENT.md and include the final URL here.]
Versioned archival (DOI)	[TODO: Archive the first release in Zenodo and include the resulting DOI here.]
Serialisations	Turtle, RDF/XML, JSON-LD, N-Triples (pre-generated by WIDOCO)
HTML documentation	WIDOCO 1.4.25 with WebVOWL; content negotiation via .htaccess under w3id-redirect/
Human-readable reference	ONTOLOGY-REFERENCE.md (class Governance bridge: (4. DAIMO-native) skos:example, identity criteria, OntoClean, tags) and VALIDATION-MATRIX.md
Release metadata	CITATION.cff, .zenodo.json, CHANGELOG.md
Validation package	validate.py, reasoner_check.py, coop_check.py
Maintenance governance	tests/negative_test.py CONTRIBUTING.md (LOT Phase 4: issue triage, pull requests, SemVer, deprecation policy)

V — Evaluation and reproducibility (5)

1. CQ-V1: Shared evaluation context of a given evaluation.
2. CQ-V2: Under a shared evaluation context, highest-metric model for a metrics via content negotiation.
3. CQ-V3: Peak alignment models under the same evaluation context and metric.
4. CQ-V4: For a given model, which benchmark suites has been evaluated on?
5. CQ-V5: Reproducibility artefacts backing an evaluation.

3. CQ-R3: Licence and usage policy of a model.
4. CQ-R4: I/O contract for the invocation interface of a published model.
5. CQ-R5: For each offering, what concrete ParticipantRole subclass does the providing agent hold? (requires subClassOf+)

D — Discovery and selection (4)

1. CQ-D1: Which models solve a given task in a given application domain?
2. CQ-D2: Which of those models are usable under a non-commercial policy pattern?
3. CQ-D3: Which models expose a service endpoint, and what authentication method does each require?
4. CQ-D4: Models reaching a minimum metric threshold under a shared evaluation context.

E — Execution and auditability (5)

1. CQ-E1: For a given catalog offering, what service endpoint, authentication, and I/O contract apply to invoking the underlying model? (requires entailment)
2. CQ-E2: For a given model deployment, which runs have been executed, by which agents, and under which execution authorisation?
3. CQ-E3: Implementation, algorithm, flow, and compute infrastructure used in a run.
4. CQ-E4: Audit evidence (hash, signer, timestamp) supporting a given run.
5. CQ-E5: Derived artefacts produced by a run, and under which authorisation.

G — Governance bridge: (4. DAIMO-native)

1. CQ-G1: Offerings that include a given model.
2. CQ-G2: Deployments serving a model, with infrastructure and I/O contract.
3. CQ-G3: Authorisation (and agreement) that authorised a specific run.
4. CQ-G4: Full provenance bundle for a derived artefact across participant contexts.

SHACL shapes

The DAIMO SHACL layer contains nine structural nodal shapes, three conformance shapes over reused classes, and six cross-class SHACL-SPARQL invariants. The module shapes/daimo-shapes.ttl is itself declared as an independent owl:Ontology with its own owl:versionIRI under <https://w3id.org/pionera/daimo/shapes/>.

- **Completeness shapes (9):** AIAssetOfferingShape, ParticipantRoleShape, ModelDeploymentShape, IOContractShape, ExecutionAuthorizationShape, DerivedArtifactShape, SharedEvaluationContextShape, AuditEvidenceShape, CrossParticipantProvenanceRecordShape.
- **Conformance shapes over reused classes (3):** OfferInDAIMOShape requires odrl:assigner and odrl:target on every odrl:Offer; MachineLearningModelInDAIMOShape requires policy, title, and identifier on it6:MachineLearningModel; RunInDAIMOShape requires flow, algorithm, associated agent, and timestamp on it6:Run.
- **Cross-class invariants (6):** DerivationAuthorizationInvariant (INV-1), RunAgentAuthorizationInvariant (INV-2), DeploymentServiceInvariant (INV-3), AuthorizationTemporalInvariant (INV-4), OfferingPolicyTargetInvariant (INV-5), and OfferingAssignerInvariant (INV-6); all encoded as sh:SPARQLConstraint.

SPARQL queries

The twenty-three SPARQL queries for the competency questions of Appendix are published in `queries/queries.md`. The validator runs each query over the OWL-RL closure of the ontology-plus-example graph and verifies that each returns at least one row. Exact counts appear in Table 7. CQ-G2 returns two rows (one per endpoint of the REST/gRPC multi-protocol deployment), confirming that modelling multiple services per deployment is a legitimate pattern directly observable from SPARQL.

Glossary of abbreviations

Table 9. Abbreviations used in this article.

Abbrev.	Expansion
AI Act	Regulation (EU) 2024/1689 on artificial intelligence
BFO	Basic Formal Ontology
CQ	Competency Question
DAIMO	Data-space AI Model Ontology
DCAT	Data Catalog Vocabulary
DCAT-AP	DCAT Application Profile
DL	Description Logic
DOI	Digital Object Identifier
DPV	Data Privacy Vocabulary
DSP	Dataspace Protocol
EDC	Eclipse Dataspace Components
FAIR	Findable, Accessible, Interoperable, Reusable
FOAF	Friend Of A Friend vocabulary
GFO	General Formal Ontology
I/O	Input / Output
LOT	Linked Open Terms methodology
MLDCAT-AP	Machine Learning Data Catalogue Application Profile
OOPS!	Ontology Pitfall Scanner
ODRL	Open Digital Rights Language
OWL	Web Ontology Language
OWL-RL	OWL 2 RL profile / rule-based reasoning
PROV-O	PROV Ontology
RDF	Resource Description Framework
RDFS	RDF Schema
REST	Representational State Transfer
SHACL	Shapes Constraint Language
SKOS	Simple Knowledge Organization System
SPARQL	SPARQL Protocol and RDF Query Language
SPDX	Software Package Data Exchange
SWJ	Semantic Web Journal
TTL	Turtle (RDF serialisation)
UFO	Unified Foundational Ontology
WIDOCO	Wizard for Documenting Ontologies

<https://w3id.org/pionera/daimo/shapes/> and the alignment module under <https://w3id.org/pionera/daimo/align> (the latter declaring `owl:imports` on the base ontology). The example graph uses the local namespace <https://example.org/daimo-scenario/>, clearly separated from the canonical space to avoid confusion between demonstration identifiers and ontology terms. Content negotiation resolves the same identifier to HTML, Turtle, RDF/XML, JSON-LD, or N-Triples depending on the Accept header; the corresponding `.htaccess` configuration is prepared under `w3id-redirect/.htaccess` pending integration in `perma-id/w3id.org`.

URI conventions

DAIMO uses the canonical namespace <https://w3id.org/pionera/daimo>, with term identifiers of the form <https://w3id.org/pionera/daimo#<Term>> and versioned IRIs under <https://w3id.org/pionera/daimo/<version>>, linked to each other through `owl:priorVersion`. The satellite modules are independent ontologies with their own versioned IRIs: the SHACL layer under